Introduction

Programming: Everyday Decision-Making Algorithms

LECTURER
Dr. Nils Roemer, External Lecturer

AFFILIATION
Planningio GmbH

Today in one sentence

In this course you learn how computer scientists approach everyday decisions, and how to turn these ideas into Python code.

Why it matters

More and more business decisions rely on data and algorithms. You don't need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker. No prior knowledge required.

Roadmap

1. About this course: topics, format and how you pass
2. How to learn programming
3. Setting up Python

About this course: topics, format and how you pass

Six topics, one per session

- **I:** Optimal Stopping
- **II:** Explore & Exploit
- **III:** Caching
- **IV:** Scheduling
- **V:** Randomness
- **VI:** Computational Kindness

Take part, it pays off

- **Participate actively**: it makes the course easier and more fun
- Inspired by the book Algorithms to Live By[1]
- All material is online: courses.nilsroemer.com

Each session: lecture, then tutorial

- **Lecture:** we discuss an algorithm for everyday decisions
- **Tutorial:** step-by-step assignments, solved together in groups

[1]Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.

- **Difficulty:** depends on your programming experience

> Tip
>
> No worries, we will help you out if you have any questions!

How you pass

- **Pass/fail course** without exams
- 75% attendance required
- **Two assignments** on algorithmic problems from the course
- **Short final presentation** of your prototype in the last lecture
- You can work in groups of **up to three students**

How to hand in assignments

- Each group submits **one solution** per assignment
- Hand in your **working notebooks**
- Due by the **end of the quarter**, earlier is better
- At least **50%** of the solutions have to be correct

You learn Python in the tutorials

We will mostly **not cover Python during the lectures!**

Question: Does anybody know why?

- The best way to learn is by **doing**
- Lectures focus on the decision-making algorithms
- You learn Python by doing the tutorials

Programming is like a new language

- At first it might be a little bit overwhelming
- First you get used to it and learn words
- Later you apply it and see results
- Important: *Practice, practice, practice!*

What you take home

- The basics of programming
- **Algorithmic thinking**: how computer scientists approach a problem
- Methods you can **apply** to practical problems

Why Python?

- **Simple syntax**: easy to read and to learn
- **Versatile**: web, data analysis, artificial intelligence
- **Big community**: help and documentation everywhere

AI is allowed, but learn yourself

- You are allowed to use AI, we use it as well
- Assistants such as Claude, ChatGPT or Gemini are **great for learning Python**

> ⚠ Warning
>
> But don't let them *replace* your learning.

How to learn programming

Roadmap

1. About this course: topics, format and how you pass
2. How to learn programming
3. Setting up Python

Three habits that work

1. Be present: attend the lecture and solve the tutorials
2. Put in some work: repeat code and try to understand it
3. Do coding: run code, play around, modify, and solve

Don't give up!

- Programming is problem solving, errors are normal
- Expect to **stretch** your comfort zone

Setting up Python

Roadmap

1. About this course: topics, format and how you pass
2. How to learn programming
3. Setting up Python

VS Code is where you write code

- Like **Word for text**, but for code: an app to **write, run and find mistakes** in code
- Apps like this are called IDEs (Integrated Development Environments)
- We use VS Code for the **whole course**

Install VS Code

- Download from code.visualstudio.com, for **Windows and Mac**
- Open it once: you should see a **welcome page**

> Tip
>
> Every step is in the Installation Guide, with a check at the end.

Add the Python and Jupyter extensions

- Out of the box, VS Code doesn't know Python. Extensions **teach it**
- Click the **Extensions** icon on the left (four small squares)
- Install **Python** and **Jupyter**, both by Microsoft

Create your course folder

- One folder for everything, called `programming-course`
- In your **home folder**, not in OneDrive
- In VS Code: **File → Open Folder...** and select it

The terminal: typing instead of clicking

- A window where you **type commands** and the computer answers with text
- Open it in VS Code: **Terminal → New Terminal**
- Copy a command from the guide, paste it, press **Enter**, wait until the line is back

Install `uv`

- `uv` installs Python and everything Python needs, you install it **once**

macOS:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Then **close VS Code completely**, open it again, and check: `uv --version`

Install Python

- Everyone uses the **same version**, so the code works the same for all

```
uv python install 3.13
```

> Tip
>
> If the installation does not work, **let us know!**

Turn your folder into a Python project

In the terminal, inside `programming-course`, one after the other:

```
uv init
```

```
uv python pin 3.13
```

Check: `uv run python main.py` prints `Hello from programming-course!`

Add notebook support

```
uv add --dev ipykernel
```

```
uv add jupyter
```

- The first lets notebooks use **the Python of your project**, the second adds the **notebook tools**

Your first notebook

- **Cmd + Shift + P** (Mac) or **Ctrl + Shift + P** (Windows), then `Create: New Jupyter Notebook`
- **Select Kernel** in the top right: **.venv** with **Python 3.13**
- Type `print("Hello")` and press **Shift + Enter**

> Tip
>
> Something not working yet? Ask us, or see “Something doesn’t work?” in the guide.

Download a tutorial and open it

- Open a tutorial on the website and click **Jupyter** under **Other Formats**
- Move the file into `programming-course`
- Open it in VS Code and choose the **.venv** kernel

> Tip
>
> That was the hardest part today!

What we covered

About this course: topics, format and how you pass

Six topics, each with a lecture and a Python session. You pass with 75% attendance, two assignments and a final presentation.

How to learn programming

Programming is learned by doing: small steps, regular practice, and don’t give up.

Setting up Python

Your laptop is ready: VS Code, Python with uv and notebooks.

Why it matters

More and more business decisions rely on data and algorithms. You don’t need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker.

Learn by doing.

Any questions

so far?

After the break: Optimal Stopping

- The “Secretary Problem” and the 37% rule
- When to stop searching and make a decision
- Testing the rule ourselves with a small simulation

> ℹ Note
>
> **That’s it for our introduction!**
> Let’s have a short break and then continue with our first topic.

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[2]

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O’Reilly. Free online

[2] The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

> **Note**
>
> Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

More Literature

For more interesting literature, take a look at the literature list of this course.